

CPSC 335, Algorithm Engineering – Spring 2026

Project 2 Part A: Greedy versus Exhaustive

Student Learning Objectives:

Demonstrate your ability to

- Translate descriptions of algorithms into pseudocode
- Analyze your pseudocode mathematically
- Implement your algorithm in C++
- Test your implementation
- Describe your results
- Observe timings correspond to Big-O trends
- Translate requirements into solutions in a timely manner

In this project you will implement and compare two algorithms that solve the same problem. The first is a greedy algorithm with a fast (i.e. polynomial) running time, while the second is an exhaustive search algorithm with a slow (i.e. exponential) running time. Both algorithms solve the problem of maximizing the amount of calories one can obtain from food products from a food pantry. This experiment will test the following hypotheses:

1. Exhaustive search algorithms are feasible to implement and produce correct outputs.
2. Algorithms with exponential running times are extremely slow, probably too slow to be of practical use.

Description - The Weight-Limited Calorie-Maximization Problem:

A food pantry has many different food products available for free to hungry people, and each product has a certain weight (in ounces) and a total calorie content. Suppose you are given a set of these products. You need to design algorithms to pick a subset of food products that fit within a given total weight while maximizing the number of calories one can obtain by eating them. You need to gain as many calories as possible from the food you obtain, but you need to carry everything by yourself so you cannot get everything you want. You need to maximize the amount of calories from the food items you can carry to stave off hunger.

There are two algorithms to help you pick a subset of the available food items, so that you can maximize the calories while keeping the total weight low enough for you to carry.

Our theory predicts that the $O(2^n \cdot n)$ exhaustive search algorithm will be far slower than the greedy algorithm with its $O(n^2)$ or $O(n \log n)$ time complexity. Your experiment will show whether this is the case.

The *Weight-Limited Calorie-Maximization problem* is:

Input:

1. A positive total weight W (integer number of ounces); and
2. A collection F of $0 < n < 64$ “food item” objects where each object $f = (w, c)$ is an integer weight $w > 0$ in ounces and a number of calories $c \geq 0$

Output:

A collection K of food items drawn from F , such that the total sum of K 's weight is within the prescribed total weight W and the total sum of K 's calories is maximized. In other words:

1. A collection $K \subseteq F$ such that:
 - a. $(\sum_{(w,c) \in K} w) \leq W$; and
 - b. $\max(\sum_{(w,c) \in K} c)$

Algorithm 1: The greedy pattern

The first algorithm uses the greedy pattern. Given the input specified above, the greedy heuristic is to partition the set of food items into a visited part and an unvisited part. Choose the “best” (highest calorie content divided by the weight) food item from the unvisited part that still fits within the budget W , then move that chosen food item to the visited part. Keep selecting the next best food items until you either cannot add more food items without exceeding your weight budget, or there are no remaining food items. Return a collection K of selected food items as specified above.

The time complexity of the greedy algorithm depends on the data structures that are used to implement it. A naive approach using unsorted vectors and sequential searches to find f takes $O(n^2)$ time. This is acceptable but not ideal. Using a heap, binary search tree, or sorting algorithm in a fairly straightforward way can speed this up to $O(n \log_2 n)$.

Algorithm 2: The exhaustive search pattern

The second algorithm uses the exhaustive search pattern. Given the input specified above, the exhaustive search heuristic is to examine every possible “viable” (total weight is within the budget W) candidate and select the candidate with the “best” (highest calorie content). Return a collection K of selected food items as specified above.

As discussed in sections 7.5.4 and 7.5.5 of ADITA, candidate subsets of F must be implemented inline using bitwise operations. The time complexity of this algorithm is expected to be $O(2^n \cdot n)$.

What to Do

You may work in groups of up to 2 people, but you must deliver individually. The Project Report must be individually handwritten by each group member (No, you can’t deliver your groupmates’ handwritten report as your own.)

Project Report: Produce and deliver a brief **handwritten** report scanned into **PDF format**. Your handwriting must be absolutely and perfectly legible to earn credit. Treat this not as work in progress, but rather your final, polished, college-level quality delivery to your customer. Submit your PDF file to Canvas. You can submit just once. Part B becomes available only after you submit Part A. Your Part B’s algorithm’s implementation must match your pseudocode as presented in this report, so get your pseudocode right before attempting to implement. Your report must include the following:

1. Algorithm 1: The greedy pattern
 - a. Design a greedy pattern algorithm for the above problem. Write detailed, clear, countable, correct, Python-like pseudocode.
 - b. Compute the step count and provide your Time Estimation function $T(n) = \dots$
 - i. Label each line of pseudocode in the left margin with its step count. Annotate the scope of any loops.
 - ii. Harvest those step counts into open-form notation (i.e., show your summations $T(n) = \sum(\dots)$)
 - iii. Show your work as you reduce that into closed-form notation citing the reduction formulas used from ADITA sections 5.2
 - iv. Evaluate your function for 3 unique values for n showing the resulting step count
 - c. Analyze your algorithm mathematically. Write a detailed, step-by-step proof proving a specific Big-O efficiency class. Clearly identify from section 3.7 of ADITA which proof methodology you are using.
2. Algorithm 2: The exhaustive search pattern
 - i. Same as above, but for the exhaustive search pattern